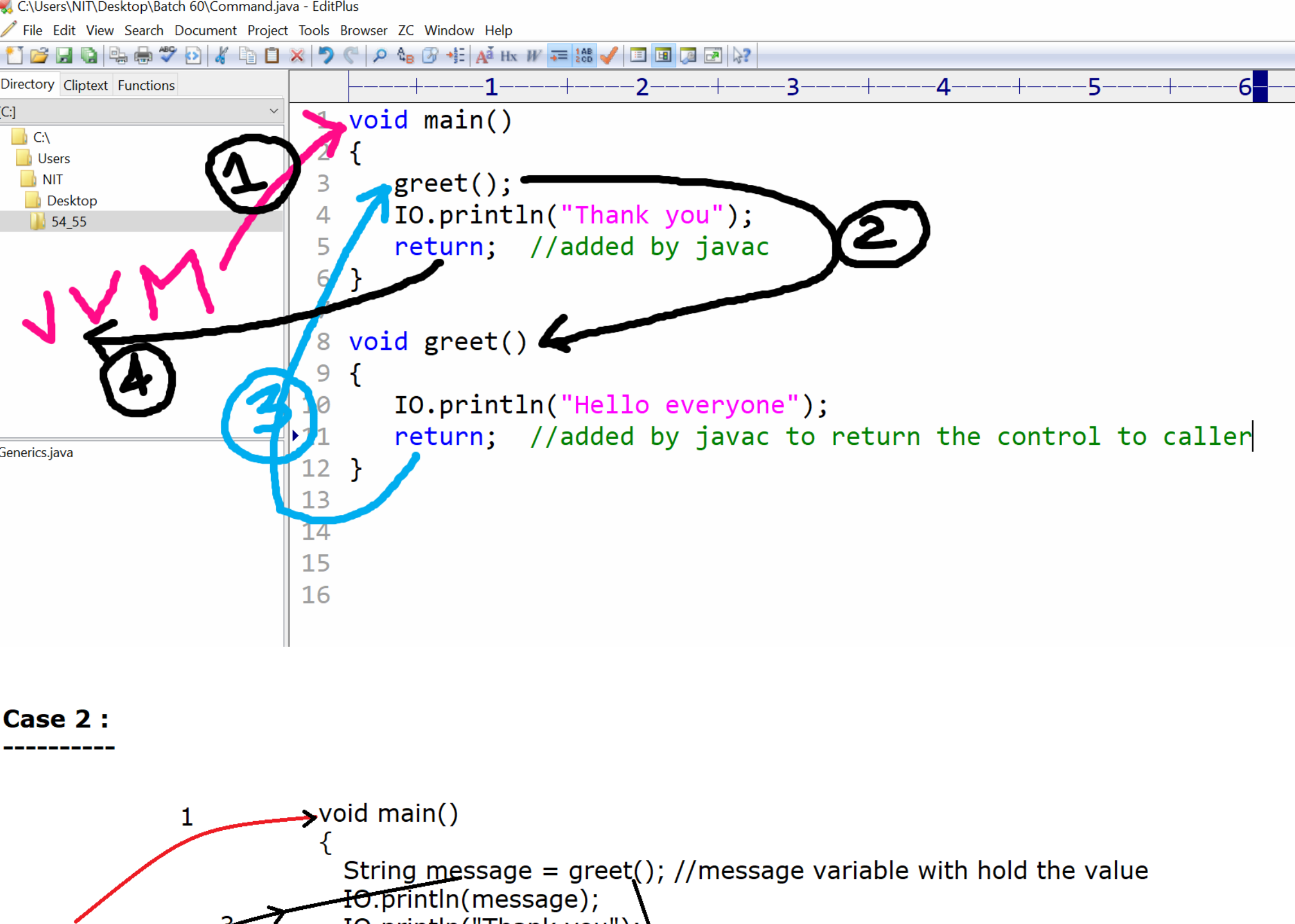
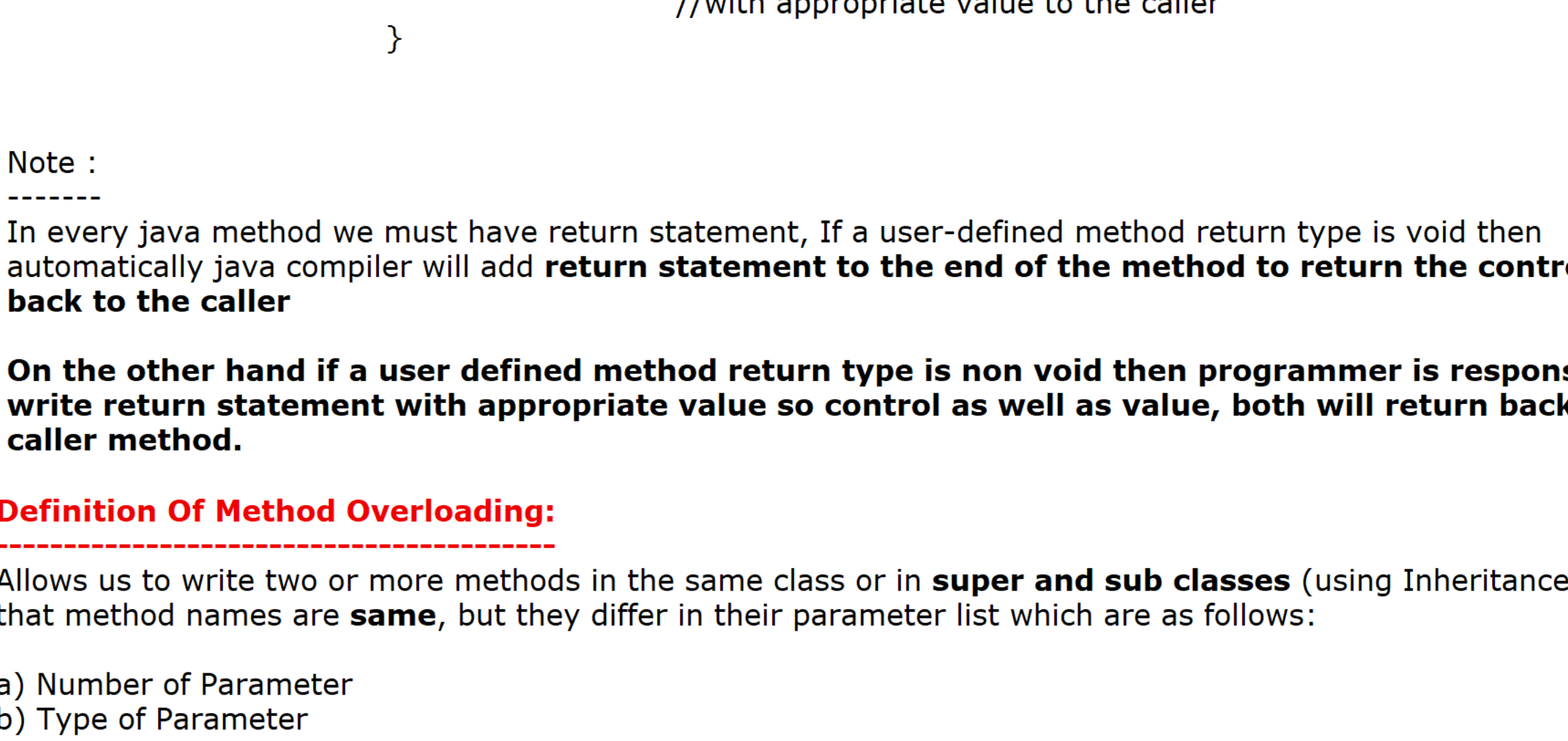


Method Execution flow with return value :

Case 1 :



Case 2 :



Note :
In every java method we must have return statement, If a user-defined method return type is void then automatically java compiler will add **return statement to the end of the method to return the control back to the caller**

On the other hand if a user defined method return type is non void then programmer is responsible to write return statement with appropriate value so control as well as value, both will return back to the caller method.

Definition Of Method Overloading:

Allows us to write two or more methods in the same class or in **super and sub classes** (using Inheritance) such that method names are **same**, but they differ in their parameter list which are as follows:

- a) Number of Parameter
- b) Type of Parameter
- c) Order of Parameter (Sequence of data type)

Example:

- a) Number of Parameter
public void display(int a) { }
public void display(int a, int b) { }
- b) Type of Parameter
public void show(int a) { }
public void show(double a) { }
- c) Order of Parameter (Sequence of data type)
public void test(int a, double b) { }
public void test(double a, int b) { }

Rules of Overloading a Method:

- 1. Methods must have the same name.
- 2. Must differ in number, type, or order of parameters.
- 3. Can change the return type.
- 4. Access modifiers can change.
- 5. Static methods can be overloaded.
- 6. Type promotion applies automatically. [Widening only, narrowing is the developer responsibility]
- 7. Cannot overload by **only changing return type**.

WAP to show MOL based on the number of parameters :

MOL1.java

```
void main()  
{  
    var result = add(12,90);  
    IO.println("Sum of two integer is :"+result);  
  
    result = add(10,20,30);  
    IO.println("Sum of three integer is :"+result);  
}  
  
public int add(int x, int y)  
{  
    return x+y;  
}  
  
public int add(int x, int y, int z)  
{  
    return x+y+z;  
}
```